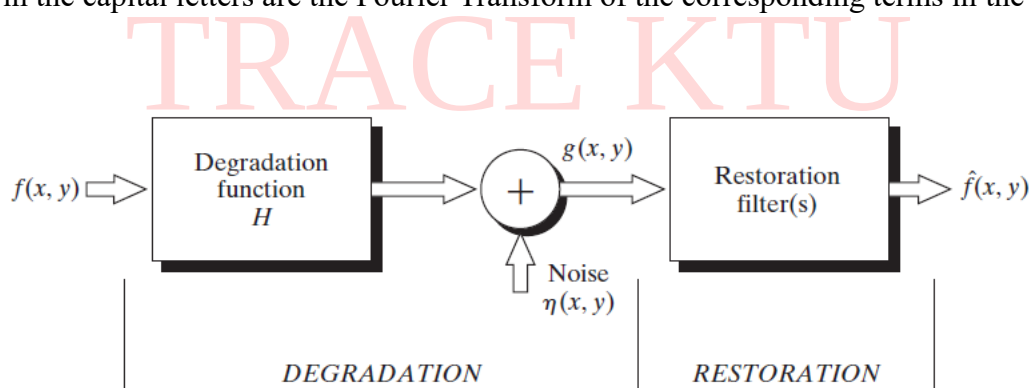


Module – 4 Image Restoration & Image Segmentation

IMAGE DEGRADATION MODEL

- Degradation process operates on a degradation function that operates on an input image with an additive noise term.
- Input image is represented by using the notation $f(x,y)$, noise term can be represented as $\eta(x,y)$.
- These two terms when combined gives the result as $g(x,y)$.
- If we are given $g(x,y)$, some knowledge about the degradation function H or J and some knowledge about the additive noise term $\eta(x,y)$, the objective of restoration is to obtain an estimate $\hat{f}(x,y)$ of the original image.
- We want the estimate to be as close as possible to the original image.
- The more we know about h and η , the closer $\hat{f}(x,y)$ will be to $f(x,y)$.
- If it is a linear position invariant process, then degraded image is given in the spatial domain by $g(x,y)=f(x,y)*h(x,y)+\eta(x,y)$
- $h(x,y)$ is spatial representation of degradation function and symbol $*$ represents convolution.
- In frequency domain we may write this equation as $G(u,v)=F(u,v)H(u,v)+N(u,v)$
- The terms in the capital letters are the Fourier Transform of the corresponding terms in the spatial domain.



NOISE MODELS

- The principal source of noise in digital images arises during image acquisition and /or transmission.
- The performance of imaging sensors is affected by a variety of factors, such as environmental conditions during image acquisition and by the quality of the sensing elements themselves.
- Images are corrupted during transmission principally due to interference in the channels used for transmission.
- Since main sources of noise presented in digital images are resulted from atmospheric disturbance and image sensor circuitry, following assumptions can be made i.e. the noise model is spatial invariant (independent of spatial location).
- The noise model is uncorrelated with the object function.

Spatial and Frequency Properties of Noise:

Spatial Properties: Spatial properties of noise refer to how the noise is distributed and manifests in the spatial domain of an image. Some common spatial properties of noise include:

- Randomness:** Noise is often characterized by random variations in pixel values. Random noise appears as random fluctuations or variations in brightness across the image.
- Spatial Correlation:** Noise can exhibit spatial correlation, meaning that neighboring pixels have similar noise characteristics. This can result in patterns or structures in the noise, such as stripes or bands.
- Clustering:** In some cases, noise may exhibit clustering behavior, where noise values tend to concentrate in specific regions of an image. This can result in localized patches or clusters of noise.
- Impulsiveness:** Impulsive noise, such as salt-and-pepper noise, appears as isolated bright or dark pixels scattered randomly throughout the image.

Frequency Properties: Frequency properties of noise refer to the distribution of noise energy across different frequencies in the frequency domain. Some common frequency properties of noise include:

- High-Frequency Noise:** Noise that has a significant amount of energy in high-frequency components appears as fine-grained or granular noise. High-frequency noise can result in a loss of fine details and a decrease in image clarity.
- Low-Frequency Noise:** Noise that exhibits energy in low-frequency components appears as large-scale variations or patterns. Low-frequency noise can manifest as uneven illumination or background variations in the image.
- Color Noise:** In color images, noise can exhibit different frequency properties in different color channels. For example, noise may have higher energy in the blue channel compared to the red and green channels, resulting in color artifacts.

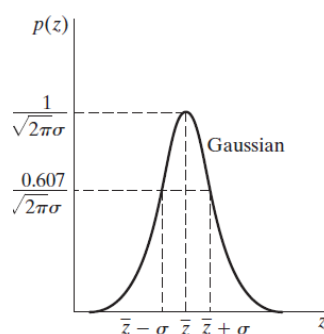
Some Important Noise Probability Density Functions

Gaussian noise

Because of its mathematical tractability in both the spatial and frequency domains, Gaussian (also called normal) noise models are used frequently in practice. In fact, this tractability is so convenient that it often results in Gaussian models being used in situations in which they are marginally applicable at best.

The PDF of a Gaussian random variable, z , is given by:
$$p(z) = \frac{1}{\sqrt{2\pi}\sigma} e^{-(z-\bar{z})^2/2\sigma^2} \quad \text{where}$$

z represents intensity, $\bar{z}$ represents mean value of z and σ is its standard deviation. The standard deviation square is called variance.

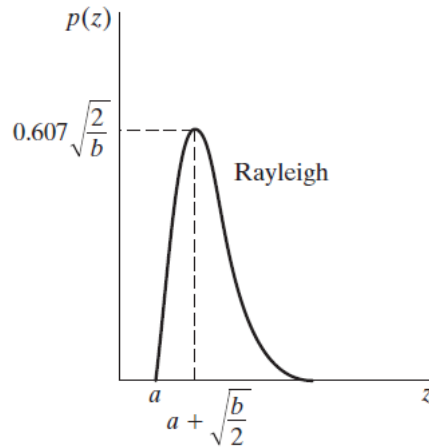


When Gaussian noise is added to an image, it manifests as a random variation in pixel intensities. The noise introduces a subtle grainy appearance, affecting the overall quality and clarity of the image. Higher levels of Gaussian noise can obscure fine details, reduce contrast, and degrade image fidelity.

Rayleigh noise

The PDF of Rayleigh noise is given by
$$p(z) = \begin{cases} \frac{2}{b}(z - a)e^{-(z-a)^2/b} & \text{for } z \geq a \\ 0 & \text{for } z < a \end{cases}$$

The mean and standard deviation of this function is given by $\bar{z} = a + \sqrt{\pi b/4}$ and $\sigma^2 = \frac{b(4 - \pi)}{4}$

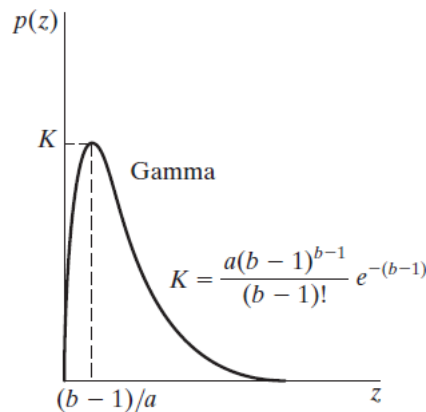


Note the displacement from the origin and the fact that the basic shape of this density is skewed to the right. The Rayleigh density can be quite useful for approximating skewed histograms.

Erlang (gamma) noise

The PDF of Erlang noise is given by
$$p(z) = \begin{cases} \frac{a^b z^{b-1}}{(b-1)!} e^{-az} & \text{for } z \geq 0 \\ 0 & \text{for } z < 0 \end{cases}$$

Where the parameters are such that $a > 0$, b is a positive number. The mean and variance of this distribution is given by $\bar{z} = \frac{b}{a}$ and $\sigma^2 = \frac{b}{a^2}$



Its shape is similar to Rayleigh disruption. This equation is referred to as gamma density it is correct only when the denominator is the gamma function.

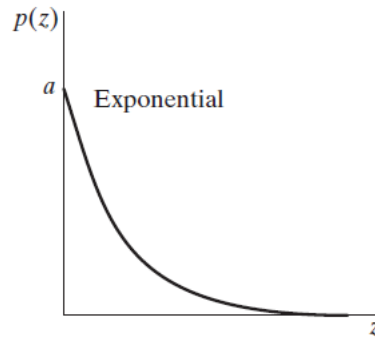
Although Eq. (5.2-5) often is referred to as the *gamma density*, strictly speaking this is correct only when the denominator is the gamma function, When the denominator is as shown, the density is more appropriately called the *Erlang density*.

Exponential noise

The PDF of *exponential* noise is given by
$$p(z) = \begin{cases} ae^{-az} & \text{for } z \geq 0 \\ 0 & \text{for } z < 0 \end{cases}$$

The mean and variance of this function is: $\bar{z} = \frac{1}{a}$ and $\sigma^2 = \frac{1}{a^2}$

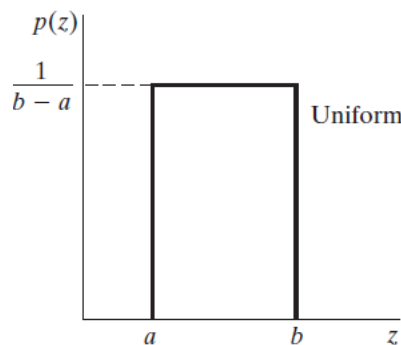
This PDF is a special case of the Erlang PDF, with $b = 1$.



Uniform noise

The PDF of *uniform* noise is given by
$$p(z) = \begin{cases} \frac{1}{b-a} & \text{if } a \leq z \leq b \\ 0 & \text{otherwise} \end{cases}$$

The mean and variance is given by $\bar{z} = \frac{a+b}{2}$ and $\sigma^2 = \frac{(b-a)^2}{12}$



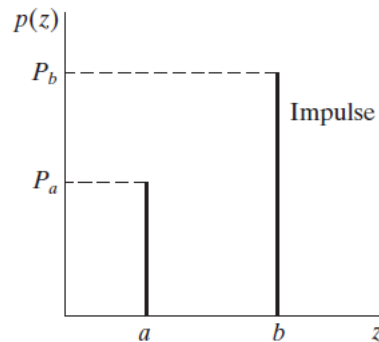
TRACE KTU

Impulse (salt-and-pepper) noise

❖ The PDF of (*bipolar*) *impulse* noise is given
$$p(z) = \begin{cases} P_a & \text{for } z = a \\ P_b & \text{for } z = b \\ 0 & \text{otherwise} \end{cases}$$

- ❖ If $b > a$ intensity b will appear as a light dot in the image.
- ❖ Conversely, level a will appear like a dark dot.
- ❖ If either P_a or P_b is zero, impulse noise is known as unipolar.
- ❖ If neither probability is zero, and especially if they are approximately equal, impulse noise values will resemble salt-and-pepper granules randomly distributed over the image.
- ❖ For this reason, bipolar impulse noise also is called *salt and- pepper* noise. *Data-drop-out* and *spike* noise also are terms used to refer to this type of noise.
- ❖ Noise impulses can be negative or positive.
- ❖ Scaling usually is part of the image digitizing process. Because impulse corruption usually is large compared with the strength of the image signal, impulse noise generally is digitized as extreme (pure black or white) values in an image.

- ❖ Thus, the assumption usually is that a and b are “saturated” values, in the sense that they are equal to the minimum and maximum allowed values in the digitized image.
- ❖ As a result, negative impulses appear as black (pepper) points in an image. For the same reason, positive impulses appear as white (salt) noise.



Periodic Noise

- ❖ Periodic noise in an image arises typically from electrical or electromechanical interference during image acquisition.
- ❖ Periodic noise can be reduced significantly via frequency domain filtering.

MEAN FILTERS:

1. Arithmetic Mean filter:

- ❖ It is the simplest mean filter.
- ❖ Let S_{xy} represents the set of coordinates in the sub - image of size $m*n$ centered at point (x,y) .
- ❖ The arithmetic mean filter computes the average value of the corrupted image $g(x,y)$ in the area defined by S_{xy} .
- ❖ The value of the restored image f at any point (x,y) is the arithmetic mean computed using the pixels in the region defined by S_{xy} .
- ❖ In other words:
$$\hat{f}(x, y) = \frac{1}{mn} \sum_{(s,t) \in S_{xy}} g(s, t)$$
- ❖ This operation can be implemented using a spatial filter of size $m * n$ in which all coefficients have value $1/mn$.
- ❖ A mean filter smooths local variations in an image, and noise is reduced as a result of blurring.

2. Geometric mean filter

- ❖ An image restored using a geometric mean filter is given by the expression
$$\hat{f}(x, y) = \left[\prod_{(s,t) \in S_{xy}} g(s, t) \right]^{\frac{1}{mn}}$$
- ❖ Here, each restored pixel is given by the product of the pixels in the sub-image window, raised to the power $1/mn$.
- ❖ a geometric mean filter achieves smoothing comparable to the arithmetic mean filter, but it tends to lose less image detail in the process.

3. *Harmonic mean filter*

- ❖ The *harmonic mean* filtering operation is given by the expression
$$\hat{f}(x, y) = \frac{mn}{\sum_{(s,t) \in S_{xy}} \frac{1}{g(s, t)}}$$
- ❖ The harmonic mean filter works well for salt noise, but fails for pepper noise.
- ❖ It does well also with other types of noise like Gaussian noise.

4. *Contraharmonic mean filter*

- ❖ The *contraharmonic mean* filter yields a restored image based on the expression
$$\hat{f}(x, y) = \frac{\sum_{(s,t) \in S_{xy}} g(s, t)^{Q+1}}{\sum_{(s,t) \in S_{xy}} g(s, t)^Q}$$
 where Q is called the *order* of the filter.
- ❖ This filter is well suited for reducing or virtually eliminating the effects of salt-and-pepper noise.
- ❖ For positive values of Q , the filter eliminates pepper noise.
- ❖ For negative values of Q , it eliminates salt noise.
- ❖ It cannot do both simultaneously.
- ❖ Note that the contraharmonic filter reduces to the arithmetic mean filter if $Q = 0$, and to the harmonic mean filter if $Q = -1$.

ORDER STATISTIC FILTERS:

- ❖ Order-statistic filters are nonlinear spatial filters whose response is based on ordering (ranking) the pixels contained in the image area encompassed by the filter, and then replacing the value of the center pixel with the value determined by the ranking result.

1. *Median Filters:*

- ❖ The best-known filter in this category is the *median filter*, which, as its name implies, replaces the value of a pixel by the median of the intensity values in the neighborhood of that pixel (the original value of the pixel is included in the computation of the median).
$$\hat{f}(x, y) = \text{median}_{(s,t) \in S_{xy}} \{g(s, t)\}$$
- ❖ Median filters are quite popular because, for certain types of random noise, they provide excellent noise-reduction capabilities, with considerably less blurring than linear smoothing filters of similar size.
- ❖ Median filters are particularly effective in the presence of *impulse noise*, also called *salt-and-pepper noise* because of its appearance as white and black dots superimposed on an image.
- ❖ Median filters are particularly effective in the presence of both bipolar and unipolar impulse noise.

2. *Max and min filters:*

- ❖ Although the median filter is by far the order-statistic filter most used in image processing, it is by no means the only one.
- ❖ The median represents the 50th percentile of a ranked set of numbers, but you will recall from basic statistics that ranking lends itself to many other possibilities.

- ❖ For example, using the 100th percentile results in the so-called *max filter*, given by $\hat{f}(x, y) = \max_{(s,t) \in S_{xy}} \{g(s, t)\}$
- ❖ This filter is useful for finding the brightest points in an image. Also, because pepper noise has very low values, it is reduced by this filter as a result of the max selection process in the sub image area S_{xy}
- ❖ The 0th percentile filter is the *min filter*: $\hat{f}(x, y) = \min_{(s,t) \in S_{xy}} \{g(s, t)\}$
- ❖ This filter is useful for finding the darkest points in an image. Also, it reduces salt noise as a result of the min operation.

3. Midpoint filter

The midpoint filter simply computes the midpoint between the maximum and minimum values in the area encompassed by the filter:

$$\hat{f}(x, y) = \frac{1}{2} \left[\max_{(s,t) \in S_{xy}} \{g(s, t)\} + \min_{(s,t) \in S_{xy}} \{g(s, t)\} \right]$$

Note that this filter combines order statistics and averaging. It works best for randomly distributed noise, like Gaussian or uniform noise.

4. Alpha Trimmed Filter:

Suppose that we delete the $d/2$ lowest and the $d/2$ highest intensity values of $g(s, t)$ in the neighborhood S_{xy} . Let $g_r(s, t)$ represent the remaining $mn - d$ pixels. A filter formed by averaging these remaining pixels is called an *alpha-trimmed mean filter*:

$$\hat{f}(x, y) = \frac{1}{mn - d} \sum_{(s,t) \in S_{xy}} g_r(s, t) \quad (5.3-11)$$

where the value of d can range from 0 to $mn - 1$. When $d = 0$, the alpha-trimmed filter reduces to the arithmetic mean filter discussed in the previous section. If we choose $d = mn - 1$, the filter becomes a median filter. For other values of d , the alpha-trimmed filter is useful in situations involving multiple types of noise, such as a combination of salt-and-pepper and Gaussian noise.

ADAPTIVE FILTERS:

1. Adaptive, local noise reduction filter

- ❖ The simplest statistical measures of a random variable are its mean and variance.
- ❖ These are reasonable parameters on which to base an adaptive filter because they are quantities closely related to the appearance of an image.
- ❖ The mean gives a measure of average intensity in the region over which the mean is computed, and the variance gives a measure of contrast in that region.
- ❖ Our filter is to operate on a local region, S_{xy} . The response of the filter at any point (x, y) on the which the region is to be centered is to be based on four quantities:
 - a. $G(x, y)$ the value of noise at (x, y)
 - b. σ^2_{η} , the variance of noise corrupting $f(x, y)$ to form $g(x, y)$

- c. m_L the local mean of pixels in S_{xy}
- d. σ_L^2 the local variance of pixels in S_{xy} .

- ❖ If σ_η^2 is zero, then the filter should return simply the value of $g(x, y)$. This is the trivial, zero noise case in which $g(x, y)$ equal to $f(x, y)$.
- ❖ If the local variance is high relative to σ_η^2 then the filter would return a value close to $g(x, y)$. A high typical local variance is associated with these edges and these would be preserved.
- ❖ If the two variances are equal, we want the filter to return the arithmetic mean value of the pixels in S_{xy} . This condition occurs when the local area has the same properties as the overall image, and local noise is to be reduced simply by averaging.
- ❖ An adaptive expression for obtaining $\hat{f}(x, y)$ based on these assumptions may be written as

$$\hat{f}(x, y) = g(x, y) - \frac{\sigma_\eta^2}{\sigma_L^2} [g(x, y) - m_L] \quad (5.3-12)$$

- ❖ The only quantity that needs to be known or estimated is the variance of the overall noise σ_η^2 . The other parameters are computed from the pixels in S_{xy} at each location (x, y) on which the filter window is centered.
- ❖ A tacit assumption in Eq. (5.3-12) is that $\sigma_\eta^2 \leq \sigma_L^2$

2. Adaptive Median Filter

- ❖ adaptive median filtering can handle impulse noise with probabilities larger than these.
- ❖ An additional benefit of the adaptive median filter is that it seeks to preserve detail while smoothing non impulse noise, something that the “traditional” median filter does not do.
- ❖ the adaptive median filter also works in a rectangular window area S_{xy} .
- ❖ Unlike those filters, however, the adaptive median filter changes (increases) the size of S_{xy} during filter operation, depending on certain conditions listed in this section.
- ❖ Keep in mind that the output of the filter is a single value used to replace the value of the pixel at (x, y) , the point on which the window S_{xy} is centered at a given time.

Consider the following notation:

- $z_{\min}$ = minimum intensity value in S_{xy}
- $z_{\max}$ = maximum intensity value in S_{xy}
- z_{med} = median of intensity values in S_{xy}
- z_{xy} = intensity value at coordinates (x, y)
- $S_{\max}$ = maximum allowed size of S_{xy}

The adaptive median-filtering algorithm works in two stages, denoted stage *A* and stage *B*, as follows:

Stage *A*:
 $A1 = z_{\text{med}} - z_{\text{min}}$
 $A2 = z_{\text{med}} - z_{\text{max}}$
 If $A1 > 0$ AND $A2 < 0$, go to stage *B*
 Else increase the window size
 If window size $\leq S_{\text{max}}$ repeat stage *A*
 Else output z_{med}

Stage *B*:
 $B1 = z_{xy} - z_{\text{min}}$
 $B2 = z_{xy} - z_{\text{max}}$
 If $B1 > 0$ AND $B2 < 0$, output z_{xy}
 Else output z_{med}

- ❖ The key to understanding the mechanics of this algorithm is to keep in mind that it has three main purposes: to remove salt-and-pepper (impulse) noise, to provide smoothing of other noise that may not be impulsive, and to reduce distortion, such as excessive thinning or thickening of object boundaries.
- ❖ The values z_{max} and z_{min} are considered statistically by the algorithm to be “impulse-like” “noise” components, even if these are not the lowest and highest possible pixel values in the image.

BASIC EDGE DETECTION

The image gradient and its properties:

The tool of choice of finding edge strength and direction at location (x, y) of an image f , the gradient is

denoted by ∇f and defined as the vector: $\nabla f = \text{grad}(f) = \begin{bmatrix} g_x \\ g_y \end{bmatrix} = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix}$.

This vector has the important property that it points in direction of greatest rate of change f at location (x, y) .

The magnitude of ∇f denoted as $M(x, y)$ is given by $\sqrt{g_x^2 + g_y^2}$ is the value of rate of change in the direction of gradient vector.

The direction of gradient vector is given by the angle $\alpha(x, y) = \tan^{-1}\left(\frac{g_y}{g_x}\right)$ measured with respect to x axis.

The direction of edge at an arbitrary point (x, y) is orthogonal to the direction $\alpha(x, y)$ of the gradient vector at that point.

Gradient Operators:

Obtaining the gradient is finding the partial derivatives $\frac{\partial f}{\partial x}$ and $\frac{\partial f}{\partial y}$ at every pixel location in the image.

Sobel Operator:

The Sobel operator is a commonly used edge detection algorithm in image processing. It is named after its inventor, Irwin Sobel, and is used to detect edges in an image by applying a set of convolution kernels.

The Sobel operator uses two kernels, one for detecting horizontal edges and another for detecting vertical edges. These kernels are 3×3 matrices that are convolved with the input image to obtain a filtered output image.

The horizontal Sobel kernel is:

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & -1 \end{bmatrix}$$

The vertical Sobel kernel is:

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

To apply the Sobel operator, the two kernels are separately convolved with the input image, and the magnitude of the resulting gradient is calculated at each pixel. The gradient magnitude represents the strength of the edge at that pixel.

The output image from the Sobel operator is usually further processed by thresholding or other techniques to produce a binary image of edges. This binary image can then be used for further image analysis, such as object detection or tracking.

Prewitt Operator:

The Prewitt operator is another widely used edge detection algorithm in image processing. Like the Sobel operator, it is named after its inventor, Judith Prewitt, and uses convolution kernels to detect edges in an image.

The Prewitt operator also uses two kernels, one for detecting horizontal edges and another for detecting vertical edges. These kernels are also 3x3 matrices that are convolved with the input image to obtain a filtered output image.

The horizontal Prewitt kernel is:

$$\begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

TRACE KTU

The vertical Prewitt kernel is:

$$\begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

Similar to the Sobel operator, the Prewitt operator calculates the magnitude of the gradient at each pixel by convolving the two kernels with the input image. This gradient magnitude represents the strength of the edge at that pixel.

The output image from the Prewitt operator can also be further processed by thresholding or other techniques to produce a binary image of edges. This binary image can then be used for further image analysis.

Compare Sobel and Prewitt Operators:

Feature	Sobel Operator	Prewitt Operator
Kernels	Two kernels for horizontal and vertical edge detection.	Two kernels for horizontal and vertical edge detection.
Performance	Better for detecting edges in the presence of noise.	Less sensitive to noise but may miss some edges.

Magnitude Calculation	Calculates magnitude using square root of sum of squares of horizontal and vertical gradients.	Calculates magnitude using sum of absolute values of horizontal and vertical gradients.
Edge Detection Results	Produces thicker edges with more emphasis on diagonals.	Produces thinner edges with more emphasis on horizontal and vertical directions.
Computation Time	Requires more computations due to the square root operation.	Faster due to the absence of a square root operation.

In general, the Sobel operator is more suitable for detecting edges in the presence of noise, while the Prewitt operator is less sensitive to noise but may miss some edges.

Laplacian Operator:

The Laplacian operator is another commonly used edge detection algorithm in image processing. It is a second-order derivative operator that calculates the Laplacian of the input image to detect edges.

The La-Placian operator of an image $f(x,y)$ is $\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$

This equation can be implemented using 3x3 mask as shown below:

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

- La-placian filter is a linear spatial filter.
- Apply it over the image and perform convolution operation.
- This will produce a **La-Placian image** that has grayish edge lines and other discontinuities
- All are superimposed on a dark featureless background.
- Background features can be recovered while still preserving the sharpening effect of La-placian operation by simply adding the original image and La-Placian image.
- $g(x,y) = f(x,y) + \nabla^2 f(x,y)$

Zero Crossings:

- ❖ Zero crossings are an important concept in image processing and edge detection. In the context of edge detection, a zero crossing is a point in an image where the intensity changes from positive to negative or vice versa, indicating the presence of an edge.
- ❖ Zero crossings can be detected using the Laplacian operator or other second-order derivative operators. After applying the Laplacian operator, the resulting filtered image is examined to identify regions where the intensity changes sign. These regions are then marked as zero crossings, indicating the location of an edge in the image.
- ❖ However, the Laplacian operator is sensitive to noise and may produce false positive zero crossings. To address this, the filtered image can be further processed using techniques such as non-maximum suppression or thresholding to remove false positives and retain only the true zero crossings.

- ❖ Zero crossings can also be used to extract other features from an image, such as corners or blob-like structures. By detecting regions with multiple zero crossings, it is possible to identify corners or regions with high curvature. By detecting regions with clustered zero crossings, it is possible to identify blob-like structures in the image.

Significance:

- ❖ It is significant because it allows us to identify the location of edges in an image based on the change in intensity from positive to negative or vice versa. This makes it a valuable tool for a wide range of applications, including object recognition, computer vision, and medical image analysis.
- ❖ The zero-crossing operator is typically applied after a filter has been applied to the image, such as the Laplacian operator. The filtered image is examined for points where the intensity changes sign, which indicates the presence of an edge. The zero-crossing operator can then be used to mark these points, making it easier to identify and analyze the edges in the image.
- ❖ One of the main advantages of the zero-crossing operator is that it is relatively robust to noise, which is a common problem in image processing. The operator can detect edges even in images with high levels of noise, making it a useful tool in many real-world applications.
- ❖ Another advantage of the zero-crossing operator is that it can be used to detect other features in an image besides edges. For example, it can be used to identify corners or blob-like structures by looking for clusters of zero crossings.

THRESHOLDING

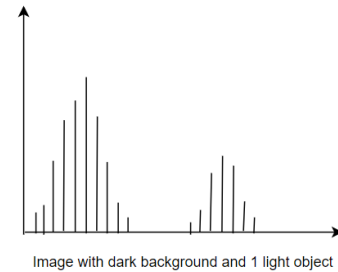
The basics of intensity thresholding

- ❖ Intensity thresholding is an image processing technique used to separate objects of interest from the background in an image. The basic idea behind thresholding is to convert an image into a binary image by setting all pixel values above a certain threshold to one and all values below the threshold to zero.
- ❖ The threshold value is typically chosen based on the intensity histogram of the image. The histogram shows the distribution of pixel intensities in the image, and the threshold can be selected at a point where there is a significant difference in the distribution between the objects of interest and the background.
- ❖ There are different types of thresholding methods, such as global thresholding, local thresholding, and adaptive thresholding. Global thresholding sets a fixed threshold value for the entire image, while local thresholding sets different threshold values for different regions of the image based on their local characteristics. Adaptive thresholding adjusts the threshold value dynamically based on the local image characteristics and is particularly useful for images with non-uniform lighting or background.
- ❖ The thresholded image can be further processed using various post-processing techniques such as morphological operations to remove small objects or fill gaps in the segmented objects.

Single value (Binary) thresholding

- Simplest thresholding method is binary thresholding.
- Here all pixel values those are greater than the specific threshold value is assign a standard value.
- Example: If the pixel value is smaller than the threshold, it is set to 0, otherwise, it is set to a maximum value (generally 255).
- This threshold value which can be changed arbitrarily.

$$g(x, y) = \begin{cases} 1, & \text{if } f(x, y) > T \\ 0, & \text{if } f(x, y) \leq T \end{cases}$$

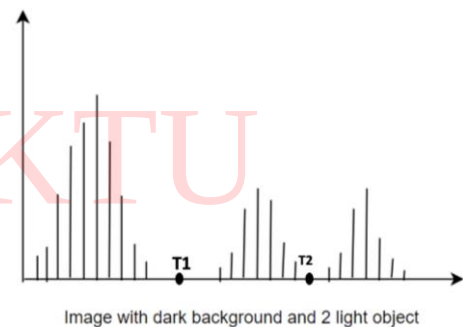


- The application of this algorithm is when we have to separate foreground with background.
- The drawback of this algorithm is that it will always segment the image into two categories.

Multi value thresholding:

- In bi-level thresholding, image is segmented into two different regions.
- The pixels with gray values greater than a certain value T are classified as object pixels, and the others with gray values lesser than T are classified as background pixels.
- Multilevel thresholding is a process that segments a gray level image into several distinct regions
- This technique determines more than one threshold for the given image and segments the image into certain brightness regions, which correspond to one background and several objects.
- The method works very well for objects with colored or complex backgrounds, on which bi-level thresholding fails to produce satisfactory results.

$$g(x, y) = \begin{cases} a, & \text{if } f(x, y) > T_2 \\ b, & \text{if } T_1 < f(x, y) \leq T_2 \\ c, & \text{if } f(x, y) \leq T_1 \end{cases}$$



BASIC GLOBAL THRESHOLDING

- ❖ **Global** → threshold T depends on the gray values of the pixel
- ❖ A global thresholding technique is the one which makes use of a single threshold value for the whole image. They are Histogram based thresholding techniques.
- ❖ The simplest of all thresholding techniques is to partition the image histogram by using a single global threshold, T.
- ❖ Creates a binary image after thresholding.
- ❖ Segmentation is then accomplished by scanning the image pixel by pixel and labeling each pixel as object or back-ground, depending on whether the gray level of that pixel is greater or less than the value of T.
- ❖ The success of this method depends entirely on how well the histogram can be partitioned.
- ❖ The objects of interest in this case are darker than the background, so any pixel with a gray level $\leq T$ was labelled black (0), and any pixel with a gray level $\geq T$ was labeled white (255).
- ❖ The key objective is to generate a binary image.

- ❖ The type of global thresholding just described can be expected to be successful in highly controlled environments.
- ❖ One of the areas in which this often is possible is in industrial inspection applications, where control of the illumination usually is feasible.

How to select the threshold T in global thresholding?

The following algorithm can be used to obtain T automatically.

1. Select an initial estimate of T
 2. Segment the image using T . This will produce two groups of pixels: G_1 consisting of all pixels with gray level values $>T$ and G_2 consisting of pixels with values $\leq T$
 3. Compute the average gray level values μ_1 and μ_2 for the pixels in regions G_1 and G_2
 4. Compute the new threshold value $T = \frac{1}{2} (\mu_1 + \mu_2)$
 5. Repeat steps 2 through 4 until the difference in T in successive iterations is smaller than a pre-defined parameter ΔT which is initialized as zero. The average intensity is the good choice for initial T value
- ❖ This simple algorithm works well in situations where there is a reasonably clear valley between the modes of the histogram related to objects and background.
 - ❖ Parameter ΔT is used to control the number of iterations in situations where speed is an important issue. In general, the larger ΔT is, the fewer iterations the algorithm will perform.
 - ❖ The initial threshold must be chosen greater than the minimum and less than maximum intensity level in the image.
 - ❖ The average intensity of the image is a good initial choice for T

OPTIMUM GLOBAL THRESHOLDING USING OTSU METHOD

- ❖ Thresholding may be viewed as a statistical-decision theory problem whose objective is to minimize the average error incurred in assigning pixels to two or more groups (also called *classes*). This problem is known to have an elegant closed-form solution known as the *Bayes decision rule*.
- ❖ The solution is based on only two parameters: the probability density function (PDF) of the intensity levels of each class and the probability that each class occurs in a given application. Unfortunately, estimating PDFs is not a trivial matter, so the problem usually is simplified by making workable assumptions about the form of the PDFs, such as assuming that they are Gaussian functions.
- ❖ Even with simplifications, the process of implementing solutions using these assumptions can be complex and not always well-suited for practical applications.
- ❖ The approach discussed in this section, called Otsu's method (Otsu [1979]), is an attractive alternative.
- ❖ The method is optimum in the sense that it maximizes the between-class variance, a well-known measure used in statistical discriminant analysis.
- ❖ The basic idea is that well-thresholded classes should be distinct with respect to the intensity values of their pixels and, conversely, that a threshold giving the best separation between classes in terms of their intensity values would be the best (optimum) threshold.

- ❖ In addition to its optimality, Otsu's method has the important property that it is based entirely on computations performed on the histogram of an image, an easily obtainable 1-D array
- ❖ The algorithm assumes that the image contains two classes of pixels: object and background. The goal is to find a threshold value that maximizes the between-class variance, which is defined as the sum of the variances of the two classes weighted by their probabilities.
- ❖ The steps involved in Otsu's method are as follows:
 1. Compute the histogram of the image and normalize it to obtain the probability density function (PDF). Denote the components of histogram p_i , $i = 0, 1, 2, \dots, L - 1$.
 2. Compute the cumulative distribution function (CDF) of the PDF, $P_1(k) = \sum_{i=0}^k p_i$ for $k = 0, 1, \dots, L - 1$.
 3. Compute the mean intensity of the image and the total variance. $m(k) = \sum_{i=0}^k i p_i$
 4. Compute global intensity mean $m_G = \sum_{i=0}^{L-1} i p_i$
 5. Compute the between-class variance for each threshold value using the class probabilities, means, and

$$\sigma_B^2(k) = \frac{[m_G P_1(k) - m(k)]^2}{P_1(k)[1 - P_1(k)]}$$

variances.

6. Obtain the Otsu threshold k^* , as the value of k for which $\sigma_B^2(k)$ is maximum. If maximum is not unique, obtain k^* by averaging the values of k corresponding to the various maxima detected.
7. Obtain the separability measure η^* by evaluating $\eta(k) = \frac{\sigma_B^2(k)}{\sigma_G^2} : k^*$

Applications:

- ❖ Otsu's method is a widely applicable technique that finds its use in numerous applications in computer vision and image processing. Some of the applications of Otsu's method are as follows:
 - **Object Detection and Recognition:** Otsu's method can effectively segment images into foreground and background regions, which makes it useful for detecting and recognizing objects in images
 - **Medical Imaging:** Otsu's method is extensively used in medical imaging applications for segmenting images of organs and tissues. Since it can assist in identifying abnormalities or anomalies in the images and help in the diagnosis
 - **Document Analysis:** Otsu's method finds its application in segmenting handwritten or printed text in document images, which can aid in optical character recognition (OCR) and document analysis
 - **Quality Control and Inspection:** Otsu's method can be used in quality control and inspection applications to detect defects or anomalies in images of manufactured products

Advantages of Otsu's method:

1. It is a simple and computationally efficient method that can be easily implemented.
2. It is an automatic thresholding method that does not require manual input or user intervention.
3. It is a widely used and well-established method that has been shown to produce good results in many applications.

4. It can handle a wide range of image types, including grayscale and color images.
5. It is a global thresholding method that can be applied to the entire image or a subset of the image.

Disadvantages of Otsu's method:

1. It assumes that the image histogram is bimodal, which may not be the case for all images. In cases where the histogram is not bimodal, Otsu's method may not produce optimal results.
2. It may not work well for images with high levels of noise or low contrast.
3. It may not be suitable for images with complex or irregular backgrounds.
4. It does not take into account spatial information or texture information, which may be important for some applications.

MULTIPLE THRESHOLDING

Otsu's method can be extended to arbitrary number of thresholds.

1. Compute the normalized histogram of the image $p_i, i=0,1,2,\dots,L-1$
2. Compute the cumulative sums $P_k = \sum_{i \in C_k} p_i, m_k = \frac{1}{P_k} \sum_{i \in C_k} i p_i, C_1, \dots, C_K$ are classes
3. Compute the cumulative mean

$$m_G = \sum_{i=1}^K P_i m_i, \sigma_B^2(k_1, \dots, k_K) = \sum_{i=1}^K P_i [m_i - m_G]^2$$

4. Compute the threshold: $k_1^*, \dots, k_{K-1}^*$ such that $\sigma_B^2(k_1^*, \dots, k_{K-1}^*) = \max_k \{\sigma_B^2(k_1, \dots, k_{K-1})\}$
5. Threshold image

$$g(x, y) = \begin{cases} a_1 & \text{if } f(x, y) \leq k_1^* \\ a_2 & \text{if } k_1^* < f(x, y) \leq k_2^* \\ \vdots & \\ a_K & \text{if } f(x, y) > k_K^* \end{cases}$$

VARIABLE THRESHOLDING

Variable thresholding is a technique used in image processing to adaptively adjust the threshold value based on the local characteristics of the image. The goal is to overcome the limitations of global thresholding, where a single threshold value is used to segment the entire image. With variable thresholding, the threshold value is adjusted based on the local intensity or texture characteristics of the image, resulting in more accurate and robust segmentation results.

Image Partitioning

- ❖ One of the simplest approaches to variable thresholding is to subdivide an image into nonoverlapping rectangles.
- ❖ This approach is used to compensate for non-uniformities in illumination and/or reflectance.
- ❖ The rectangles are chosen small enough so that the illumination of each is approximately uniform.

Variable thresholding based on local image properties

- ❖ A more general approach than the image subdivision method discussed in the previous section is to compute a threshold at every point, (x, y) in the image based on one or more specified properties computed in a neighborhood of (x, y) .

- ❖ We illustrate the basic approach to local thresholding using the standard deviation and mean of the pixels in a neighborhood of every point in an image. These two quantities are quite useful for determining local thresholds because they are descriptors of local contrast and average intensity.
- ❖ Let σ_{xy} and m_{xy} denotes the standard deviation and mean value of set of pixels contained in a neighbourhood S_{xy} centered at coordinates (x, y)
- ❖ The following are common forms of variable, local thresholds: $T_{xy} = a\sigma_{xy} + bm_{xy}$ where a and b are non- negative constants and $T_{xy} = a\sigma_{xy} + bm_G$ m_G is the global mean.
- ❖ The segmented image is computed as:
$$g(x, y) = \begin{cases} 1 & \text{if } f(x, y) > T_{xy} \\ 0 & \text{if } f(x, y) \leq T_{xy} \end{cases}$$
 where $f(x, y)$ is the input

Image. This equation is evaluated over all pixel locations and different threshold is computed at each location (x, y) using the neighborhood S_{xy} .

- ❖ Significant power (with a modest increase in computation) can be added to local thresholding by using predicates based on the parameters computed in the neighbourhoods of (x, y) :

$$g(x, y) = \begin{cases} 1 & \text{if } Q(\text{local parameters}) \text{ is true} \\ 0 & \text{if } Q(\text{local parameters}) \text{ is false} \end{cases}$$

where Q is a predicate based on the parameters
computed using the pixels in the neighbourhood S_{xy} .

Using moving averages:

- ❖ A special case of the local thresholding method just discussed is based on computing a moving average along scan lines of an image.
- ❖ This implementation is quite useful in document processing, where speed is a fundamental requirement.
- ❖ The scanning typically is carried out line by line in a zigzag pattern to reduce illumination bias.
- ❖ Let z_{k+1} represents the intensity of the point encountered in the scanning sequence at step $k+1$, then moving averages at this new point is given by:

$$m(k+1) = \frac{1}{n} \sum_{i=k+2-n}^{k+1} z_i$$

$$= m(k) + \frac{1}{n}(z_{k+1} - z_{k-n})$$

Where n denotes the number of points used in computing the
Average and $m(1) = z_1 / n$.

MULTIVARIABLE THRESHOLDING

- ❖ multivariable thresholding may be viewed as a distance computation.
- ❖ Suppose that we want to extract from a color image all regions having a specified color range: say, reddish hues.
- ❖ Let $\mathbf{a}$ denote the average reddish color in which we are interested.
- ❖ One way to segment color image based on this parameter is to compute a distance measure, $D(\mathbf{z}, \mathbf{a})$, between an arbitrary point $\mathbf{z}$ and the average color $\mathbf{a}$, then we can segment the image as follows:

$$g = \begin{cases} 1 & \text{if } D(\mathbf{z}, \mathbf{a}) < T \\ 0 & \text{otherwise} \end{cases}$$

where T is the threshold and it is understood that the distance computation is performed at all coordinates in the input image to generate the corresponding segmented values in g .

REGION BASED SEGMENTATION

Region Growing

- ❖ As its name implies, *region growing* is a procedure that groups pixels or subregions into larger regions based on predefined criteria for growth.
- ❖ The basic approach is to start with a set of “seed” points and from these grow regions by appending to each seed those neighboring pixels that have predefined properties similar to the seed (such as specific ranges of intensity or color).
- ❖ In Region growing method, we start with a random pixel as the seed pixel and then check the adjacent pixels.
- ❖ If the adjacent pixels satisfy the predefined rules, then that pixel is added to the region of the seed pixel
- ❖ Region is grown from the seed pixel by adding in neighbouring pixels that are similar, increasing the size of the region.
- ❖ When the growth of one region stops, we simply choose another seed pixel which does not yet belong to any region and start again.
- ❖ Drawback:
 - Current region dominates the growth process
 - Different choices of seeds may give different segmentation results.

- ❖ A basic region-growing algorithm based on 8-connectivity may be stated as follows.

Let: $f(x, y)$ denote an input image array; $S(x, y)$ denote a *seed* array containing 1s at the locations of seed points and 0s elsewhere; and Q denote a predicate to be applied at each location (x, y) . Arrays f and S are assumed to be of the same size. A basic region-growing algorithm based on 8-connectivity may be stated as follows.

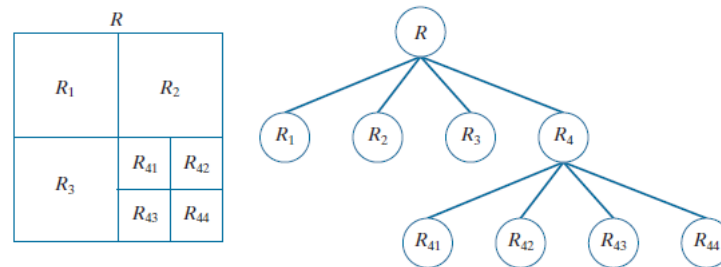
1. Find all connected components in $S(x, y)$ and erode each connected component to one pixel; label all such pixels found as 1. All other pixels in S are labeled 0.
2. Form an image f_Q such that, at a pair of coordinates (x, y) , let $f_Q(x, y) = 1$ if the input image satisfies the given predicate, Q , at those coordinates; otherwise, let $f_Q(x, y) = 0$.
3. Let g be an image formed by appending to each seed point in S all the 1-valued points in f_Q that are 8-connected to that seed point.
4. Label each connected component in g with a different region label (e.g., 1, 2, 3, ...). This is the segmented image obtained by region growing.

Region splitting and merging

An alternative is to subdivide an image initially into a set of arbitrary, disjoint regions and then merge and/or split the regions in an attempt to satisfy the conditions of segmentation.

- ❖ Let R represents the entire image region and select Q the predicate. One approach for segmentation is to subdivide it successively into smaller and smaller quadrants so that for any region R_i , $Q(R_i) = \text{TRUE}$.
- ❖ We start with the entire region. When $Q(R_i) = \text{FALSE}$, we divide the image segments into quadrants.
- ❖ If Q is false for any quadrant, we subdivide that quadrant into sub – quadrants and so on.

- ❖ This particular splitting technique has a convenient representation in the form of so called, quad trees, i.e. trees with exactly four descendants. the root of the tree corresponds to the entire image, and that each node corresponds to the subdivision of a node into four descendant nodes. In this case, only R_4 was subdivided further.
- ❖ If only splitting is used, the final partition normally contains adjacent regions with identical properties. This drawback can be remedied by allowing *merging* as well as splitting.
- ❖ Satisfying the constraints of segmentation requires merging only adjacent regions whose combined pixels satisfy the predicate Q . That is, two adjacent regions R_j and R_k are merged only if $Q(R_j \cup R_k) = \text{TRUE}$.
- ❖ The preceding discussion can be summarized by the following procedure in which, at any step, we
 1. Split into four disjoint quadrants any region R_i for which $Q(R_i) = \text{FALSE}$.
 2. When no further splitting is possible, merge any adjacent regions R_j and R_k for which $Q(R_j \cup R_k) = \text{TRUE}$.



TRACE KTU